

CS7CS2 – Optimisation for Machine Learning

Student Name: Stephen Komolafe

Student Number: 21336975

Week 8 Assignment

Q1:

Projected gradient descent was applied to the quadratic objective function:

$$f(x_1, x_2) = (x_1 - 1.2)^2 + 2(x_2 - 2.5)^2 + 0.4x_1x_2$$

under two different feasible sets,

$$X_1 = \{(x_1, x_2) \in \mathbb{R}^2: 0.5 \leq x_1 \leq 2.5, 0.5 \leq x_2 \leq 3.5\}$$

$$X_2 = \{(x_1, x_2) \in \mathbb{R}^2: x_1 \geq 0.5, x_2 \geq 0.5, x_1 \leq x_2\}.$$

The aim of this question was to derive the gradient, implement projected gradient descent, and compare how the geometry of the feasible set affects the projection step and the resulting trajectory of the iterates. In the code, the objective function, its gradient, the projection operators, and a general projected gradient descent routine were implemented using NumPy, while Matplotlib was used to generate the required plots. The same algorithmic structure was used for both feasible sets so that the numerical behaviour under X_1 and X_2 could be compared directly.

(a) Deriving $\nabla f(x_1, x_2)$

Differentiating the objective function, with respect to each variable gives:

$$\nabla f(x_1, x_2) = \begin{bmatrix} 2x_1 + 0.4x_2 - 2.4 \\ 0.4x_1 + 4x_2 - 10 \end{bmatrix}.$$

This expression was coded in the function `grad_f(x)`, which returns the gradient as a NumPy array.

```
import numpy as np
import matplotlib.pyplot as plt
def f(x):
    x1, x2 = x
    return (x1 - 1.2)**2 + 2*(x2 - 2.5)**2 + 0.4*x1*x2

def grad_f(x):
    x1, x2 = x
    return np.array([
        2*x1 + 0.4*x2 - 2.4,
        0.4*x1 + 4*x2 - 10
    ])
```

The projected gradient descent for X_1 was then implemented using the update:

$$x^{(t+1)} = P_{X_1} \left(x^{(t)} - \alpha \nabla f(x^{(t)}) \right),$$

With a constant step size $\alpha = 0.2$, an initial point $x^{(0)} = (2.4, 0.7)$, and for 60 iterations. The function `pgd()` was written in a general form so that it could later be reused for both feasible sets by changing the projection function.

```
def pgd(x0, alpha, proj, n_iters=60):
    xs = [x0.copy()]
    fs = [f(x0)]
    x = x0.copy()

    for _ in range(n_iters):
        x = proj(x - alpha * grad_f(x))
        xs.append(x.copy())
        fs.append(f(x))

    return np.array(xs), np.array(fs)
```

During the run, both the iterates and the objective values were stored. Numerically, for X_1 the method converged from $(2.4, 0.7)$ to $(0.714286, 2.428571)$, while the objective decreased from 8.592000 to 0.940000.

(b) Projection onto X_1 using elementwise clipping

The feasible set:

$$X_1 = \{(x_1, x_2) \in \mathbb{R}^2: 0.5 \leq x_1 \leq 2.5, 0.5 \leq x_2 \leq 3.5\},$$

is a rectangular box, so the Euclidean projection onto it can be performed coordinate-wise. Each component is clipped independently to its allowable interval, which was implemented using `np.clip` in the function `proj_X1()`, which maps x_1 into $[0.5, 2.5]$ and x_2 into $[0.5, 3.5]$.

```
def proj_X1(x):
    return np.array([
        np.clip(x[0], 0.5, 2.5),
        np.clip(x[1], 0.5, 3.5)
    ])
```

This works because minimising the squared Euclidean distance to a point over a Cartesian product of closed intervals separates into two independent one-dimensional minimisation problems. Since the given starting point $(2.4, 0.7)$ is already feasible for X_1 , no initial correction was required before running the iterations. The feasibility check for the final iterate showed:

$$x_1 - 0.5 = 0.214286,$$

$$2.5 - x_1 = 1.785714,$$

$$x_2 - 0.5 = 1.928571,$$

$$3.5 - x_2 = 1.071429,$$

thus, the final iterate lies strictly inside X_1 .

(c) Projected gradient descent for X_2 using repeated projections

For the second feasible set,

$$X_2 = \{(x_1, x_2) \in \mathbb{R}^2: x_1 \geq 0.5, x_2 \geq 0.5, x_1 \leq x_2\},$$

the full projection is less direct, so it was implemented through repeated projections onto the three individual constraint sets. In the code, `proj_c1()` enforces $x_1 \geq 0.5$, `proj_c2()` enforces $x_2 \geq 0.5$, and `proj_c3()` enforces $x_1 \leq x_2$.

```
def proj_c1(x):
    y = x.copy()
    y[0] = max(y[0], 0.5)
    return y

def proj_c2(x):
    y = x.copy()
    y[1] = max(y[1], 0.5)
    return y

def proj_c3(x):
    y = x.copy()
    if y[0] > y[1]:
        avg = 0.5 * (y[0] + y[1])
        y[0] = avg
        y[1] = avg
    return y
```

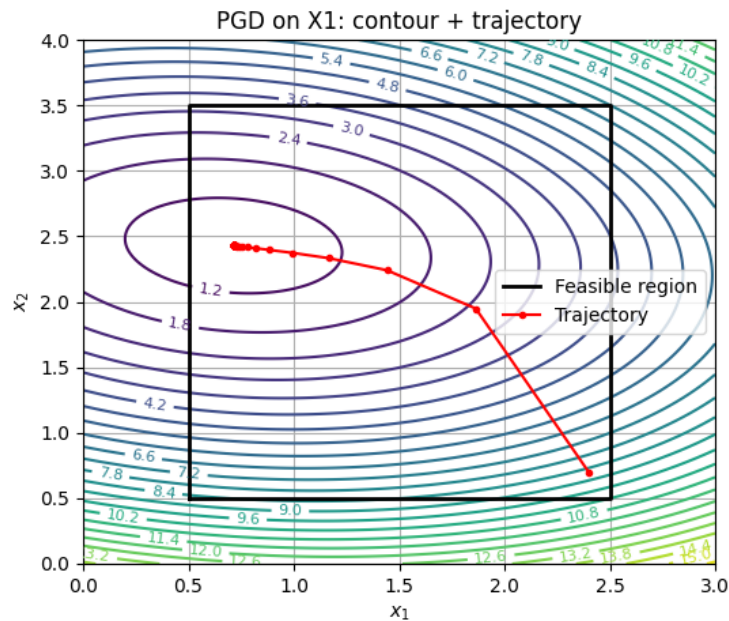
The first two are threshold operations, while the third checks whether $x_1 > x_2$ and, if so, projects orthogonally onto the line $x_1 = x_2$ by replacing both coordinates with their average. These three projections were then applied for 20 iterations in the function `proj_X2_repeat()`.

```
def proj_X2_repeat(x, num_iters=20):
    y = x.copy()
    for _ in range(num_iters):
        y = proj_c1(y)
        y = proj_c2(y)
        y = proj_c3(y)
    return y
```

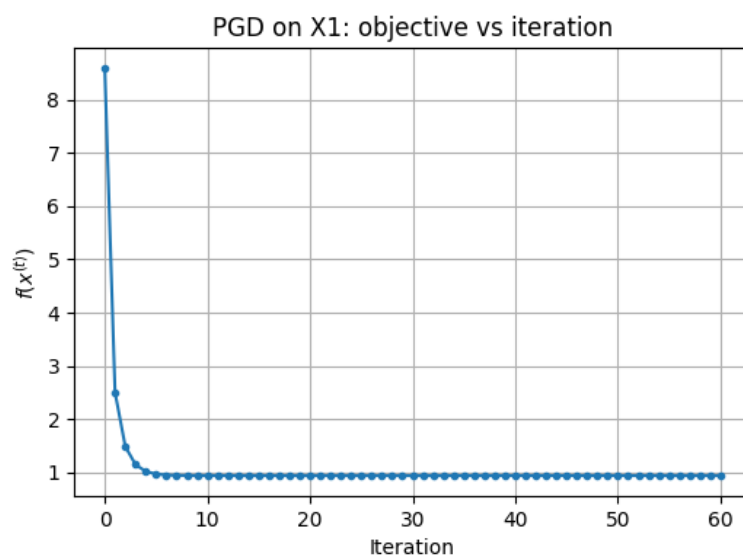
Since the original point $(2.4, 0.7)$ violates the condition $x_1 \leq x_2$, it was first projected onto X_2 , giving the feasible starting point $(1.55, 1.55)$. the code first projected it onto X_2 before applying projected gradient descent. From there, projected gradient descent converged to the same final point $(0.714286, 2.428571)$, with the objective decreasing from 2.8885 to 0.94.

(d) Plots for X_1

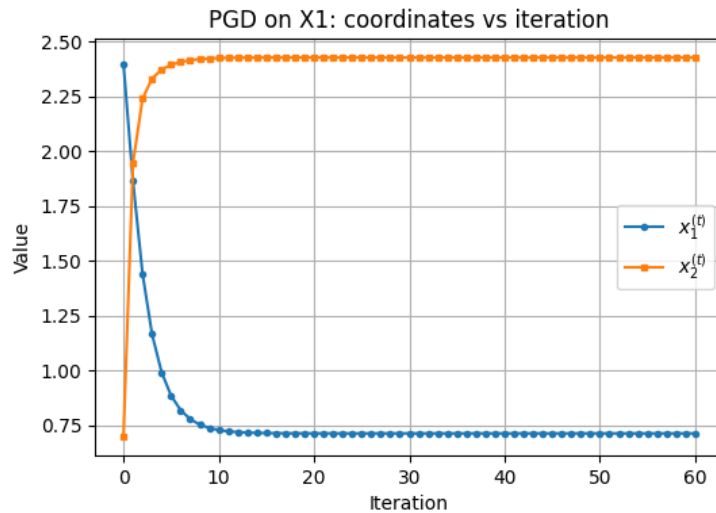
For X_1 , three plots were generated to illustrate the numerical behaviour.



The contour plot shows the trajectory starting at (2.4, 0.7) and moving smoothly across the interior of the rectangular feasible region toward (0.714286, 2.428571). Since the point is feasible from the outset and the minimiser also lies inside the box, the path is smooth and no iterate is forced onto the boundary.



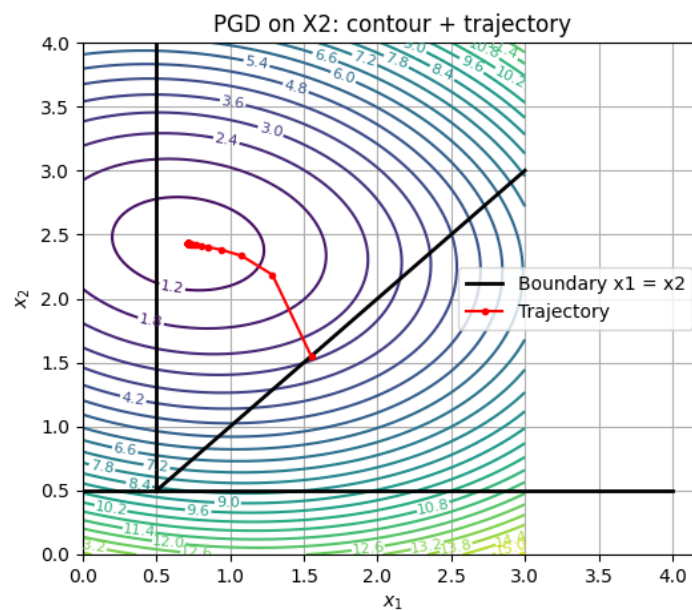
The objective-value plot confirms rapid convergence: the value falls from 8.592 initially to approximately 2.503 after one iteration, then to 1.486, 1.145, and 1.018 over the next few iterations, before stabilising at 0.94. The last several values are identical to six decimal places (0.940000), showing numerical convergence by the end of the run.



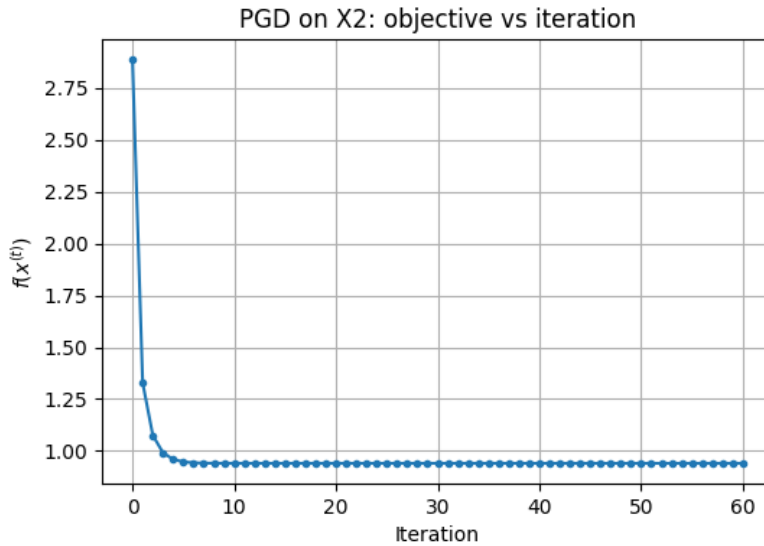
The coordinate plot shows that $x_1^{(t)}$ decreases monotonically from 2.4 to 0.714286, while $x_2^{(t)}$ increases from 0.7 to 2.428571. This indicates a direct movement toward the minimiser without oscillation.

(e) Plots for X_2

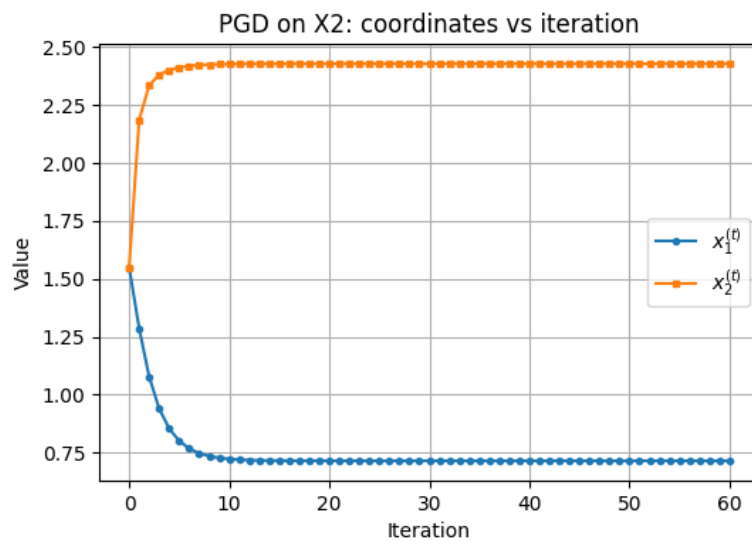
To aid in describing the projected gradient descent using repeated projections, the same three-plot structure was then produced for X_2 ,



The contour plot shows that the trajectory begins from the projected feasible point (1.55, 1.55), rather than the original (2.4, 0.7) and then moves toward the same minimiser (0.714286, 2.428571). Compared with the X_1 case, the early part of the trajectory reflects the effect of the repeated projection enforcing the condition $x_1 \leq x_2$.



The objective-value plot again shows steady convergence, decreasing from 2.8885 initially to approximately 1.3291, 1.0755, 0.9909, and 0.9593 in the first few iterations, before settling at 0.94. Since the starting point is already much closer to the minimiser than in the X_1 case, the initial objective value is substantially smaller.



The coordinate plot shows $x_1^{(t)}$ decreasing from 1.55 to 0.714286, while $x_2^{(t)}$ increases from 1.55 to 2.428571. The change is again smooth, with no long-term oscillation.

(f) Comparison of the results for X_1 and X_2

Comparing the two experiments, projected gradient descent converged to the same numerical minimiser for both feasible sets, namely (0.7143, 2.4286), with final objective value $f(x) = 0.94$.

This is expected, because the unconstrained minimiser of the quadratic objective already satisfies all the inequalities defining both X_1 and X_2 . As a result, the projection step mainly affects the early stages of the algorithm, especially for X_2 , where the

original point first had to be corrected to (1.55, 1.55). For X_1 , method started farther from the minimiser, with objective 8.592, so the total decrease was larger. For X_2 , the projected starting point was already closer, with objective 2.8885, so fewer large changes were needed. In both cases, the final iterate lies in the interior of the feasible set rather than on the boundary. For X_1 , the final margins to the constraints are all strictly positive, and for X_2 the final values:

$$x_1 - 0.5 = 0.214286, \quad x_2 - 0.5 = 1.928571, \quad x_2 - x_1 = 1.714286$$

show that all inequalities are satisfied strictly. Therefore, although the two feasible sets have different geometries and require different projection strategies, they lead to the same interior optimum for this problem.

Q2:

For this question, the goal was to handle inequality constraints using both fixed penalty methods and a primal–dual approach. The objective function was

$$f(x_1, x_2) = (x_1 - 0.2)^2 + (x_2 - 2)^2$$

with nonlinear constraints $x_1 \geq 0.5$ and $x_1 x_2 \geq 1$. The implementation was carried out in Python using NumPy for computation and Matplotlib for visualisation. Two strategies were coded: a fixed-penalty method based on a penalised objective, and a primal–dual method that updates Lagrange multipliers adaptively. The same initial point $x^{(0)} = (1.4, 0.6)$ was used throughout, and each method was run for 100 iterations to allow direct numerical comparison.

(a) Defining g_1 , g_2 , Q , and F

The constraints were rewritten in the standard form $g_i(x) \leq 0$ as:

$$g_1(x_1, x_2) = 0.5 - x_1, \quad g_2(x_1, x_2) = 1 - x_1 x_2.$$

Using these, a linear penalty function was defined as:

$$Q(x) = \lambda_1 \max(0, g_1(x)) + \lambda_2 \max(0, g_2(x)),$$

and the penalised objective became:

$$F(x) = f(x) + Q(x).$$

In the code, the functions `g1()` and `g2()` compute the constraint values, while `penalty()` implements $Q(x)$. The full penalised objective is given by `F()`. Since the penalty contains max terms, the function is not differentiable everywhere, so a sub-gradient was implemented in `subgrad_F(x, lam1, lam2)`, where each penalty term contributes only when its corresponding constraint is violated.

```

def g1(x):
    x1, x2 = x
    return 0.5 - x1

def g2(x):
    x1, x2 = x
    return 1.0 - x1*x2

def penalty(x, lam1, lam2):
    return lam1 * max(0.0, g1(x)) + lam2 * max(0.0, g2(x))

def F(x, lam1, lam2):
    return f(x) + penalty(x, lam1, lam2)

def subgrad_F(x, lam1, lam2):
    grad = grad_f(x).copy()

    if g1(x) > 0:
        grad += lam1 * grad_g1(x)

    if g2(x) > 0:
        grad += lam2 * grad_g2(x)

    return grad

```

(b) Gradient descent on F for two choices of penalty weights

Gradient descent was then applied to the penalised objective F using the update:

$$x^{(t+1)} = x^{(t)} - \alpha \nabla F(x^{(t)}),$$

with step size $\alpha = 0.08$ and 100 iterations. This was implemented in the function `gradient_descent_penalty()`, which stores the iterates, objective values, and constraint violations at each step.

```

def gradient_descent_penalty(x0, alpha, lam1, lam2, n_iters=100):
    x = x0.copy()
    xs = [x.copy()]
    vals = [F(x, lam1, lam2)]
    g1_vals = [g1(x)]
    g2_vals = [g2(x)]

    for _ in range(n_iters):
        x = x - alpha * subgrad_F(x, lam1, lam2)
        xs.append(x.copy())
        vals.append(F(x, lam1, lam2))
        g1_vals.append(g1(x))
        g2_vals.append(g2(x))

    return np.array(xs), np.array(vals), np.array(g1_vals), np.array(g2_vals)

```

Two cases were tested: a small penalty ($\lambda_1 = \lambda_2 = 0.5$) and a large penalty ($\lambda_1 = \lambda_2 = 0.4$). In the small-penalty run, the final iterate was (0.558032, 2.051341), with $g_1 = -0.058032$ and $g_2 = -0.144715$, so the final point was feasible. In the large-penalty run, the final iterate was (0.484165, 2.017593), with $g_1 = 0.015835$ and $g_2 = 0.023152$, so the final point was slightly infeasible. This shows that, for this implementation and step size, larger fixed penalties did not automatically guarantee a better final feasible point.

```

x0 = np.array([1.4, 0.6])
alpha_pen = 0.08
n_iters_pen = 100

```



```

traj_small, vals_small, g1_small, g2_small = gradient_descent_penalty(
    x0, alpha_pen, lam1=0.5, lam2=0.5, n_iters=n_iters_pen
)

traj_large, vals_large, g1_large, g2_large = gradient_descent_penalty(
    x0, alpha_pen, lam1=4.0, lam2=4.0, n_iters=n_iters_pen
)

```

(c) Primal–dual iteration

The primal–dual method was implemented using the updates:

$$x^{(t+1)} = x^{(t)} - \alpha \nabla \left(f(x) + \lambda_1^{(t)} g_1(x) + \lambda_2^{(t)} g_2(x) \right),$$

$$\lambda_i^{(t+1)} = \left[\lambda_i^{(t)} + \beta g_i(x^{(t)}) \right]_+ = \max\{0, \lambda_i^{(t)} + \beta g_i(x^{(t)})\}, \quad i = 1, 2,$$

with $\alpha = 0.06$ and $\beta = 0.08$. This was implemented for 100 iterations in the function `primal_dual()`, which stores the primal iterates, the multiplier history, the objective values, and the constraint values.

```

def primal_dual(x0, alpha=0.06, beta=0.08, n_iters=100):
    x = x0.copy()
    lam = np.array([0.0, 0.0]) # [lam1, lam2]

    xs = [x.copy()]
    lam_hist = [lam.copy()]
    vals = [f(x)]
    g1_vals = [g1(x)]
    g2_vals = [g2(x)]

    for _ in range(n_iters):
        # primal step
        grad_L = grad_f(x) + lam[0]*grad_g1(x) + lam[1]*grad_g2(x)
        x = x - alpha * grad_L

        # dual step
        lam[0] = max(0.0, lam[0] + beta * g1(x))
        lam[1] = max(0.0, lam[1] + beta * g2(x))

        xs.append(x.copy())
        lam_hist.append(lam.copy())
        vals.append(f(x))
        g1_vals.append(g1(x))
        g2_vals.append(g2(x))

    return np.array(xs), np.array(lam_hist),
           np.array(vals), np.array(g1_vals),
           np.array(g2_vals)

(...)

traj_pd, lam_pd, vals_pd, g1_pd, g2_pd = primal_dual(
    x0, alpha=0.06, beta=0.08, n_iters=100
)

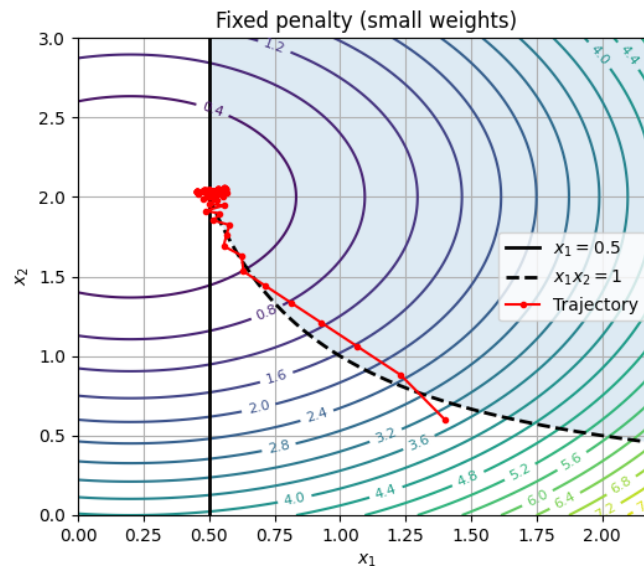
```

Starting from (1.4, 0.6), the method converged to (0.488285, 2.053437). The final multiplier values were $\lambda_1 = 0.134931$ and $\lambda_2 = 0.215608$. At the final iterate, $g_1 = 0.011715$ and $g_2 = -0.002662$, so the method ended extremely close to the feasible boundary, satisfying the second constraint and only slightly violating the first. The

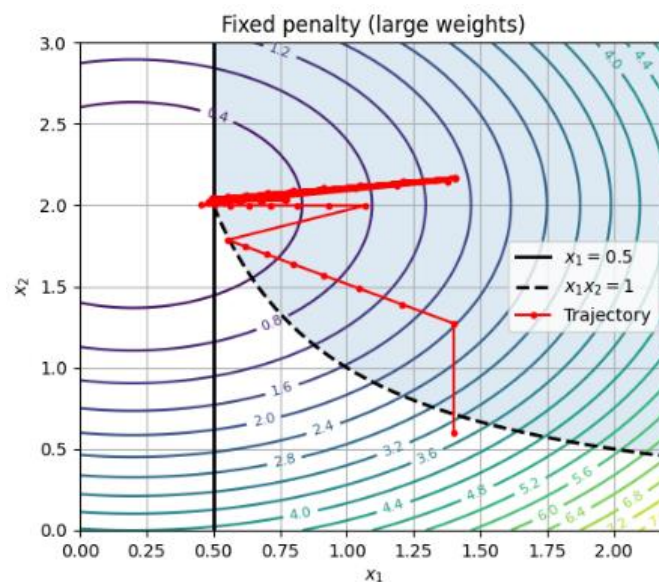
multiplier history also shows adaptive behaviour: λ_1 grew steadily near the end, while λ_2 rose earlier and then stabilised.

(d) Plots for the fixed-penalty method

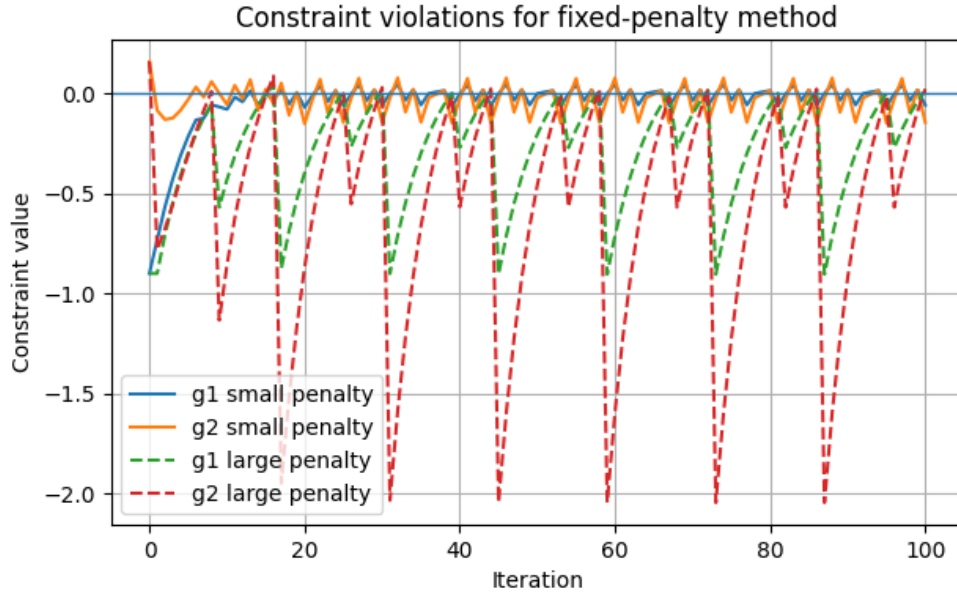
For the fixed-penalty method, three plots were generated.



The first figure shows a contour plot of the objective together with the feasible region and the trajectory for the small-penalty case. The trajectory moves from (1.4, 0.6) toward a point near (0.558, 2.051), which lies inside the feasible region.



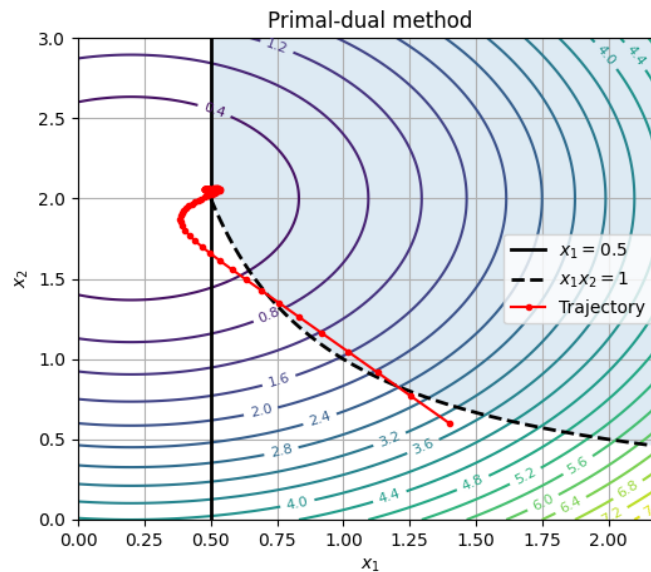
The second figure shows the corresponding contour plot for the large-penalty case. Although the path is pushed strongly toward the constraint boundaries, the final iterate (0.484165, 2.017593) ends slightly outside the feasible set.



The third figure plots $g_1(x^{(t)})$ and $g_2(x^{(t)})$ against iteration for both penalty choices. In the small-penalty run, the final values are both negative, although the late iterates oscillate near the boundary, as seen from the previous values of g_1 and g_2 . In the large-penalty run, both constraints are satisfied for much of the run but the final iterate slightly crosses into infeasibility, with $g_1 = 0.015835$ and $g_2 = 0.023152$. This indicates that the fixed-penalty method is sensitive not only to the penalty weights but also to the step size and the nonsmooth nature of the penalised objective.

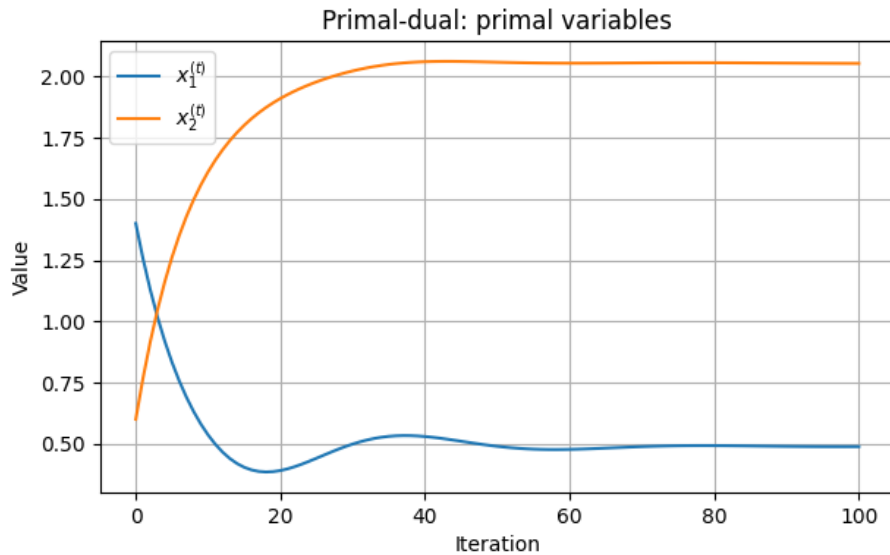
(e) Plots for the primal-dual method

For the primal-dual method, three analogous plots were generated:

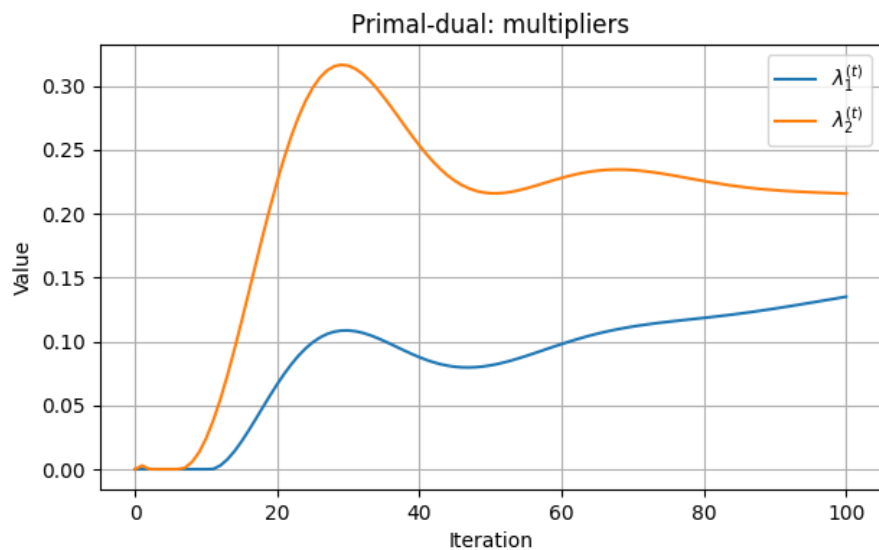


The first figure shows a contour plot of the primal-dual trajectory starting from $(1.4, 0.6)$ and moving upward and leftward toward the constrained solution near $(0.5, 2)$. Early in the run, the path mainly corrects the violation of the constraint $x_1 x_2 \geq 1$, while later

iterations move close to the vertical boundary $x_1 = 0.5$. The final iterate $(0.4883, 2.0534)$ lies very near the boundary region where both constraints are nearly active. $g_2(x) \approx -0.00266$ is satisfied and $g_1(x) \approx 0.01172$ is only slightly violated, which is consistent with the trajectory ending just beside the feasible set boundary.



The second figure plots $x_1^{(t)}$ and $x_2^{(t)}$ against iteration. The output shows that $x_1^{(t)}$ decreases from 1.4 to approximately 0.4883, while $x_2^{(t)}$ increases from 0.6 to approximately 2.0534. Both variables begin to stabilise as gets closer to the 100th iteration.



The third figure plots $\lambda_1^{(t)}$ and $\lambda_2^{(t)}$ against iteration. The multipliers start from zero, and by the end of the run $\lambda_1^{(t)}$ settles at around 0.134931, and $\lambda_2^{(t)}$ at 0.215608. In particular, λ_1 keeps increasing slightly near the end because the first constraint remains mildly violated, while λ_2 stabilises once the second constraint is approximately enforced.

(f) Comparison of fixed penalties and adaptive multipliers

Comparing the methods using the actual numerical output, the fixed-penalty method is simple to implement but quite sensitive to parameter choice and can behave non-intuitively. In this run, the small-penalty case ended at a feasible point, (0.558032, 2.051341) while the large-penalty case ended slightly infeasible at (0.484165, 2.017593). This suggests that simply increasing the penalty weight does not necessarily improve the result when using a fixed step size on a non-smooth penalised objective. The primal–dual method behaved more systematically: it converged very close to the active constraint boundary near (0.5, 2), with final point (0.488285, 2.053437), satisfying g_2 and only slightly violating g_1 . It also achieved the lowest final value of the original objective among the three methods, namely 0.085964, compared with 0.130823 for the small-penalty run and 0.237007 for the large-penalty run. The primal-dual method provided the clearest adaptive response to constraint violations through the multiplier updates, while the fixed-penalty method required more careful tuning and showed more oscillation near the boundary.

Q3:

This question compares the Frank–Wolfe algorithm with projected gradient descent on a constrained quadratic optimisation problem. The objective function:

$$f(x_1, x_2) = (x_1 - 1.5)^2 + (x_2 - 1.2)^2$$

was minimised over a convex polygon defined by linear inequalities. The implementation was carried out in Python using NumPy, with both algorithms sharing the same initial point $x^{(0)} = (2.8, 0.8)$ and similar iteration counts.

```
def f(x):  
    x1, x2 = x  
    return (x1 - 1.5)**2 + (x2 - 1.2)**2
```

(a) Feasible set, vertices, and the Frank–Wolfe linear subproblem

The feasible set is defined by the constraints $x_1 \geq 0.5$, $x_2 \geq 0.5$, $x_1 + x_2 \leq 4$, $x_1 \leq 3$, and $x_2 \leq 3$, which together form a compact convex polygon. By intersecting these constraints, the vertices of the feasible region are

$$(0.5, 0.5), (3, 0.5), (3, 1), (1, 3), (0.5, 3).$$

These vertices were stored explicitly in the code as a NumPy array vertices.

```
vertices = np.array([  
    [0.5, 0.5],
```

```
[3.0, 0.5],
[3.0, 1.0],
[1.0, 3.0],
[0.5, 3.0]
])
```

In the Frank–Wolfe algorithm, the linear subproblem minimizes a linear function over this polytope:

$$\min_{x \in X} \nabla f(x^{(t)})^T x$$

Since a linear function over a compact polytope attains its minimum at an extreme point, it is sufficient to evaluate the linear objective at all vertices and choose the best one.

(b) Deriving $\nabla f(x_1, x_2)$

Differentiating the objective function gives

$$\nabla f(x_1, x_2) = \begin{bmatrix} 2(x_1 - 1.5) \\ 2(x_2 - 1.2) \end{bmatrix}.$$

This gradient was implemented in the function `grad_f()`, which returns the two partial derivatives as a NumPy array.

```
def grad_f(x):
    x1, x2 = x
    return np.array([
        2*(x1 - 1.5),
        2*(x2 - 1.2)
    ])
```

Since the objective is smooth and quadratic, the gradient is simple to evaluate and leads to stable update rules in both Frank–Wolfe and projected gradient descent.

(c) Frank–Wolfe algorithm Implementation

The Frank–Wolfe algorithm was implemented by solving the linear subproblem at each iteration by checking all vertices of the feasible set. This was coded in the function `linear_subproblem()`, which computes $\nabla f(x^{(t)})^T v$ for each vertex v and returns the minimiser. The update rule used was:

$$x^{(t+1)} = (1 - \gamma_t)x^{(t)} + \gamma_t z^{(t)},$$

where $z^{(t)}$ is the selected vertex and $\gamma_t = \beta^t$. This was implemented in the function `frank_wolfe()` which stores the iterates, the selected vertices, and the objective values at each step.

```
def linear_subproblem(grad):
    vals = vertices @ grad
    idx = np.argmin(vals)
```

```

return vertices[idx].copy()

def frank_wolfe(x0, beta=0.8, n_iters=60):
    x = x0.copy()
    xs = [x.copy()]
    zs = []
    vals = [f(x)]

    for t in range(n_iters):
        g = grad_f(x)
        z = linear_subproblem(g)
        gamma = beta**t
        x = (1 - gamma) * x + gamma * z

        zs.append(z.copy())
        xs.append(x.copy())
        vals.append(f(x))

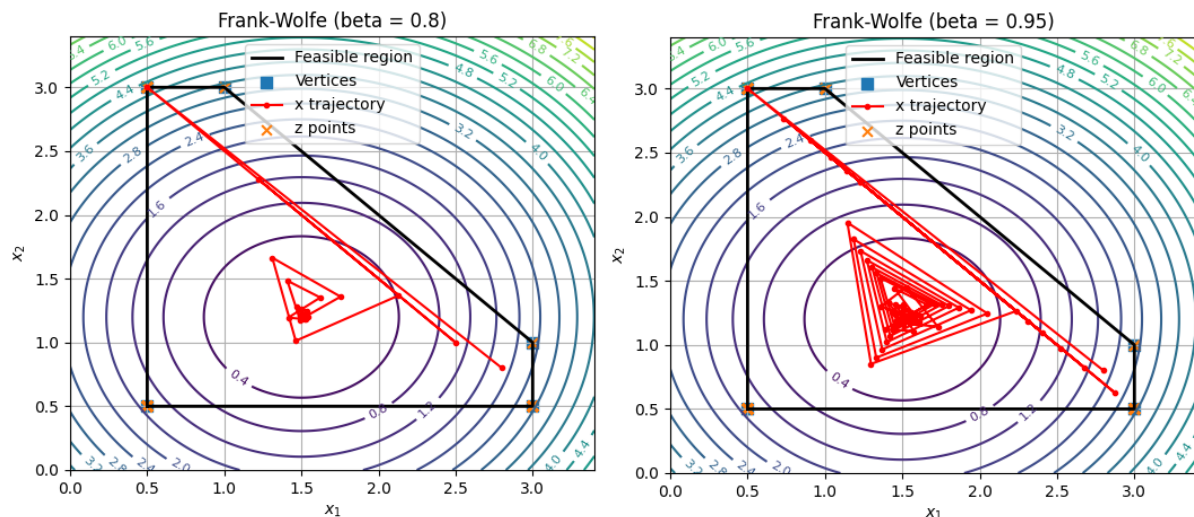
    return np.array(xs), np.array(zs), np.array(vals)

```

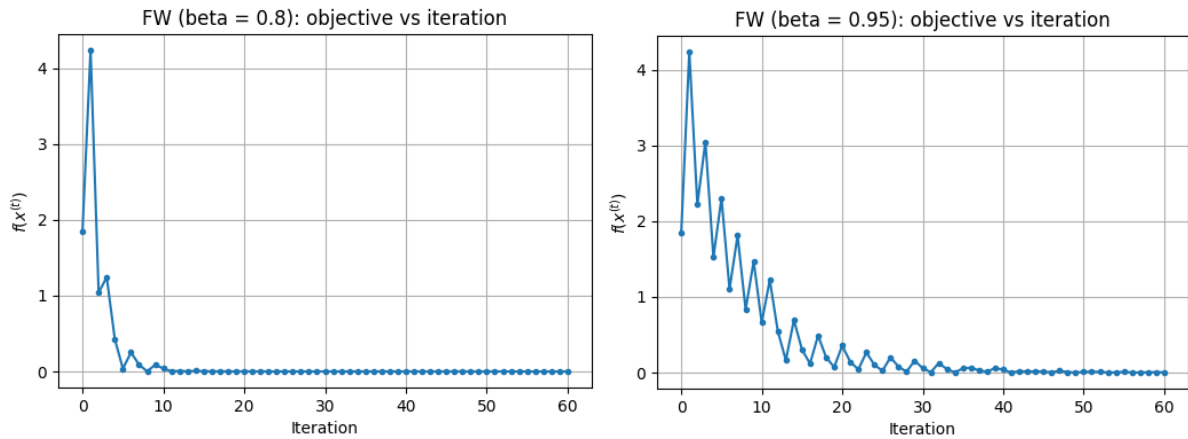
Two values of β , namely 0.8 and 0.95, were tested. For $\beta = 0.8$, the final iterate after 60 iterations was $(1.4999999, 1.20000315)$ with objective value approximately 9.93×10^{-12} , which is essentially the exact minimiser. For $\beta = 0.95$, the final iterate was $(1.45556162, 1.2324582)$ with objective value 0.0030283, so the method was still close to the solution but less accurate after the same number of iterations. This confirms that the slower-decaying step size $\beta = 0.95$ led to more persistent oscillation.

(d) Plots for Frank-Wolfe ($\beta = 0.8$ & 0.95)

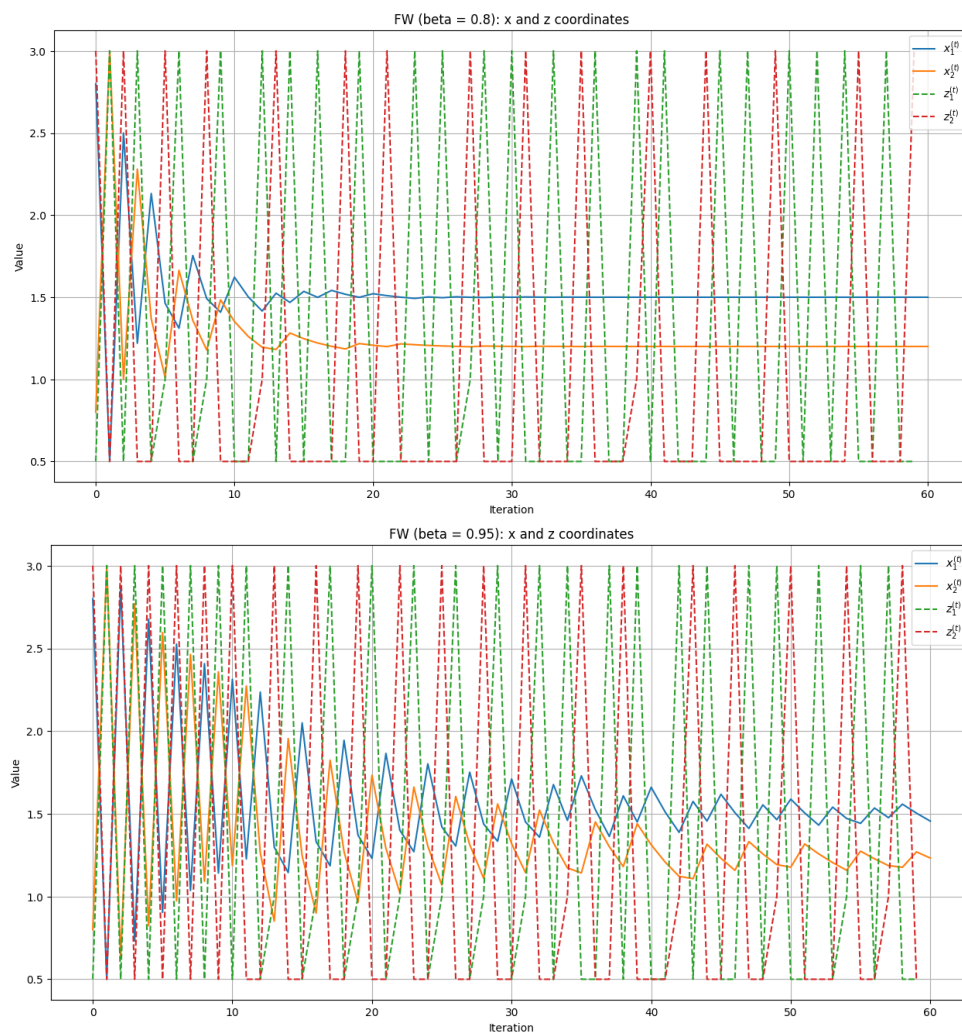
For each value of β , three plots were generated.



The contour plots show the feasible polygon, its vertices, and the iterate trajectory. In both cases, the path repeatedly moves toward different vertices before returning toward the interior minimiser. For $\beta = 0.8$, the trajectory settles much more effectively near $(1.5, 1.2)$, whereas for $\beta = 0.95$ the path continues to zig-zag more noticeably.



The objective-value plots show a clear difference in convergence behaviour. For $\beta = 0.8$, the objective eventually drops essentially to zero, although the first few values are not monotone: 1.85, 4.24, 1.04, 1.2448, 0.427055. This reflects the fact that Frank-Wolfe can temporarily move away from the minimiser when stepping toward a boundary vertex. For $\beta = 0.95$, the oscillation is stronger and persists longer, with the first few values 1.85, 4.24, 2.22125, 3.050492, and 1.525793, and a final value of 0.0030283, which is small but not as close to zero as in the $\beta = 0.8$ case.



The coordinate plots of $x^{(t)}$ and $z^{(t)}$ show the characteristic behaviour of Frank–Wolfe. The selected vertices change frequently over time, especially between (0.5, 0.5), (0.5, 3), and (3, 0.5). For $\beta = 0.8$, the most frequently selected vertices were (0.5, 0.5) and (3, 0.5), chosen 23 and 20 times respectively, and the iterates eventually stabilised very close to (1.5, 1.2). For $\beta = 0.95$, the vertex switching remained more active later in the run, which explains the less accurate final point.

(e) Projected gradient descent and numerical comparison

Projected gradient descent was then implemented for the same problem using the update:

$$x^{(t+1)} = P_X \left(x^{(t)} - \alpha \nabla f(x^{(t)}) \right),$$

with step size $\alpha = 0.2$. Since the feasible set is described by several linear inequalities, the projection was implemented by repeated projections onto the individual constraints, using the functions `proj_c1()` through `proj_c5()`, combined in `proj_X_repeat()`.

```
def proj_c1(x):
    y = x.copy()
    y[0] = max(y[0], 0.5)
    return y

def proj_c2(x):
    y = x.copy()
    y[1] = max(y[1], 0.5)
    return y

def proj_c3(x):
    # x1 + x2 <= 4
    y = x.copy()
    if y[0] + y[1] > 4:
        s = (y[0] + y[1] - 4) / 2.0
        y[0] -= s
        y[1] -= s
    return y

def proj_c4(x):
    y = x.copy()
    y[0] = min(y[0], 3.0)
    return y

def proj_c5(x):
    y = x.copy()
    y[1] = min(y[1], 3.0)
    return y

def proj_X_repeat(x, iters=20):
    y = x.copy()
    for _ in range(iters):
        y = proj_c1(y)
        y = proj_c2(y)
        y = proj_c3(y)
        y = proj_c4(y)
        y = proj_c5(y)
    return y
```

The algorithm was implemented in the function `pgd()`, which stores the iterates and objective values.

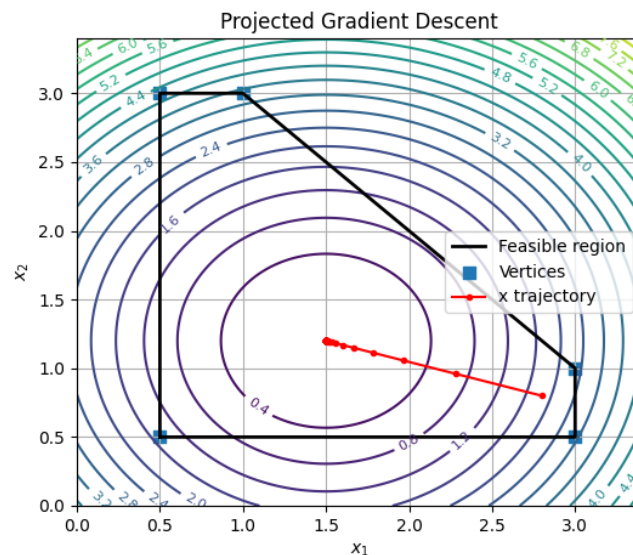
```
def pgd(x0, alpha=0.2, n_iters=60):
    x = x0.copy()
    xs = [x.copy()]
    vals = [f(x)]

    for _ in range(n_iters):
        x = proj_X_repeated(x - alpha * grad_f(x))
        xs.append(x.copy())
        vals.append(f(x))

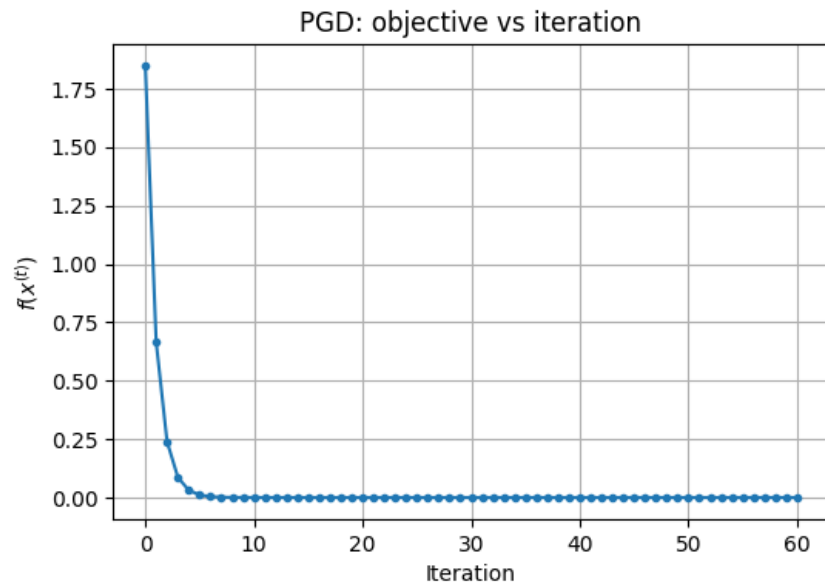
    return np.array(xs), np.array(vals)
```

Starting from (2.8, 0.8), PGD converged to (1.5, 1.2) with final objective value approximately 4.41×10^{-27} , which is numerically exact. The first few objective values were 1.85, 0.666, 0.23976, 0.086314, and 0.031073, showing a smooth and rapid decrease without the oscillatory behaviour seen in Frank–Wolfe.

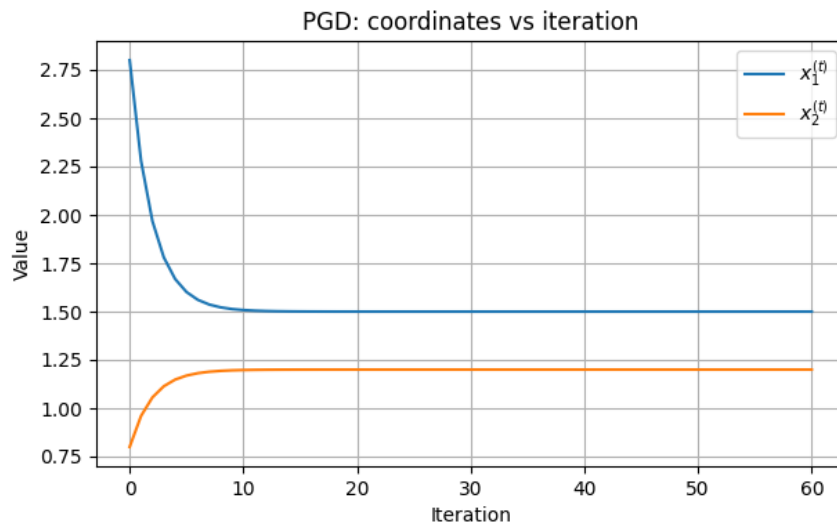
Three plots were generated for projected gradient descent:



The first figure shows a contour plot of the objective function together with the feasible region and the trajectory of the iterates. Starting from the initial point (2.8, 0.8), the trajectory moves directly toward the minimiser (1.5, 1.2) along a smooth path that remains entirely within the feasible region. Unlike the Frank-Wolfe method, there is no visible zigzagging or movement toward vertices; instead, the path follows a nearly straight descent direction toward the centre of the contours. This reflects the fact that projected gradient descent is able to move freely in the interior of the set, with projection only acting when necessary.



The second figure shows the objective value $f(x^{(t)})$ versus iteration. The plot demonstrates rapid and monotonic convergence: the objective decreases from 1.85 initially to approximately 0.666, 0.23976, 0.086314, and 0.031073 in the first few iterations, before quickly approaching zero. After around 10 iterations, the objective is essentially zero to numerical precision and remains flat thereafter. This indicates very fast convergence compared to the Frank–Wolfe method, with no oscillations or increases in the objective value.



The coordinate plot shows $x_1^{(t)}$ decreasing steadily from 2.8 to 1.5, while $x_2^{(t)}$ increases from 0.8 to 1.2, with no visible oscillation near the end.

The third figure plots the coordinates $x_1^{(t)}$ and $x_2^{(t)}$ against iteration. The plot shows that $x_1^{(t)}$ decreases smoothly from 2.8 to 1.5, while $x_2^{(t)}$ increases from 0.8 to 1.2. Both variables approach their limiting values rapidly and stabilise after approximately 10 iterations, with no oscillatory behaviour. This confirms that the method converges steadily and directly to the interior minimiser.

(f) Comparison of Frank–Wolfe and projected gradient descent

The numerical results show a clear contrast between the two methods. Since the true minimiser $(1.5, 1.2)$ lies in the interior of the feasible region, projected gradient descent is particularly well suited to this problem. It converges directly and smoothly to the solution, reaching $(1.5, 1.2)$ to numerical precision with essentially zero objective value. Frank–Wolfe, by contrast, generates search directions toward boundary vertices, so its trajectory naturally zigzags through convex combinations of extreme points before approaching the interior optimum.

In terms of boundary behaviour, Frank–Wolfe is strongly influenced by the geometry of the boundary because each step is based on a vertex solution of the linear subproblem, whereas projected gradient descent is free to move directly toward the minimiser and only uses projection to maintain feasibility. In terms of oscillation, the Frank-Wolfe runs were clearly more oscillatory, especially for $\beta = 0.95$, where the objective values and coordinates continued to fluctuate late in the run. The $\beta = 0.8$ choice performed much better and reached an almost exact minimiser but still displayed initial oscillations due to the vertex-based search directions. In terms of objective decrease, projected gradient descent decreased the objective much more smoothly and rapidly than Frank-Wolfe. Finally, in terms of implementation, Frank-Wolfe was straightforward here because the feasible set had only five vertices, making the linear subproblem easy to solve by enumeration. However, for this problem, projected gradient descent was numerically more efficient and provided the most direct path to the interior optimum.

All final iterates were found to lie in the interior of the feasible set. For example, for the Frank-Wolfe run with $\beta = 0.8$, the final margins to the constraints were:

$$x_1 - 0.5 = 1.000000, x_2 - 0.5 = 0.700003, 4 - x_1 - x_2 = 1.299997,$$

with the upper-bound margins also strictly positive, and the same was true for the $\beta = 0.95$ and PGD runs. Thus, although Frank–Wolfe operates through boundary vertices, both methods ultimately converged to interior points in this problem.